

PartialSpoof 부분 위조 음성 DSP 기반 탐지 · 국소화

중간 정리 (단순 로지스틱 회귀 베이스라인)

과제: 음성 일부에 삽입된 짧은 가짜(TTS/VC) 구간을 DSP로 탐지·국소화

데이터: PartialSpoof database v1.2 (dev subset)

현재 단계: DSP 전용/딥러닝 모델 이전 — 3개 DSP 피처 + 단순 로지스틱 회귀

저장소: github.com/leeytkfng/PS-Detect-and-locate

작성: leeytkfng

주의: 본 보고서의 결과는 '단순 회귀(logistic regression)' 수준의 베이스라인입니다. 아직 PartialSpoof 논문의 SSL/딥러닝 대응책(CM)이나 전용 DSP 모델을 구현한 것이 아니며, '간단한 특징만으로도 부분 위조 단서가 실제로 검출되는가'를 확인하는 단계입니다.

1. 과제 개요 — Partial Spoof란

Partial Spoof(PS, 부분 위조)는 진본(bonafide) 음성 발화 안에 TTS(음성합성)/VC(음성변환)로 만든 짧은 가짜 구간을 삽입·치환하는 공격이다. 단어/음절 하나만 바뀌어도 문장의 의미가 뒤집힐 수 있어(예: 'not' 삽입), 전체가 가짜인 기존 스푸핑보다 탐지가 어렵다.

- 탐지(detection): 발화 안에 가짜 구간이 하나라도 있는가? (발화 단위 판정)
- 국소화(localization): 가짜가 '어디(몇 초~몇 초)'인가? (구간/프레임 단위 판정)
- 기존 대응책(CM)은 통째로 가짜인 데이터로 학습되어 PS에 성능이 급락 -> PS 전용 접근 필요.

본 과제 접근: 논문은 공격자가 아티팩트를 고급 처리로 지우지 않고 기본 DSP만 쓴다고 가정한다. 따라서 연결 이음새(concatenation seam)·스펙트럼 불연속 같은 흔적이 남고, 이를 DSP 특징으로 잡을 수 있다는 것이 우리의 출발 가설이다.

2. 데이터셋 — PartialSpoof dev

ASVspoof2019 LA를 재료로 구축. dev subset만 사용(train/eval은 5~6GB라 후순위). 발화 24,844개(16kHz). 두 종류의 라벨이 있다.

2.1 발화(utterance) 라벨 — protocols/

형식: <화자id> <발화id> - <시스템> <라벨>

예: LA_0069 CON_D_0000001 - CON spoof

라벨	발화 수	id 접두어	의미
bonafide	2,548	LA_D_*	원본 진짜 음성
spoof	22,296	CON_D_*	가짜 구간이 삽입된 음성

2.2 구간(segment) 라벨 — segment_labels/ (가장 중요)

문서에는 텍스트(<id> <dur> <label> start-end-...)로 적혀 있었지만, 실제 v1.2는 .npy 파일이다. 구조: 0-d numpy object -> dict{발화id: 프레임별 '0'/'1' 배열}. 규약: 1 = bonafide, 0 = spoof. 프레임 수 = ceil(길이 / 해상도). 6+1개 해상도(0.01 ~ 0.64초)별로 별도 파일이 존재한다.

검증 결과: con_wav / dev.lst / protocol / segment_labels 네 출처의 개수가 24,844 = 2,548(bonafide) + 22,296(spoof)로 완전히 일치. 라벨-오디오 정합성 확인 완료.

3. 라벨 구조 확인 — 해상도 × 타입, 오디오↔라벨 매칭

해상도가 고울수록(0.64→0.01s) 가짜 구간 경계가 정밀해진다. 또한 '완전 가짜'는 해상도 의존적이다: 0.16s 기준으로는 완전가짜처럼 보이던 발화도, 10ms(0.01s)로 보면 앞뒤에 진본 프레임(무음·비음성)이 남아 대부분 '부분 가짜'로 분류된다(0.01s에서 완전가짜는 단 1개).

[그림 누락: compare_resolutions.png]

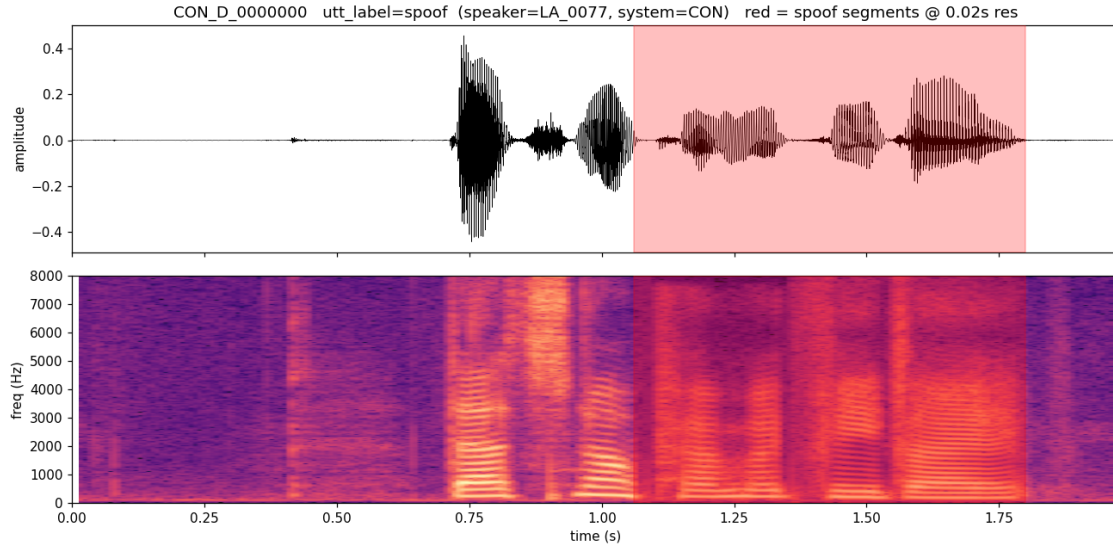


그림 2. 오디오↔라벨 매칭(CON_D_0000000, 0.02s). 빨간 음영=가짜 구간(1.06~1.80s). ~1.0s의 끊김이 연결 이음새.

4. DSP 단서 측정값 — '어색함'의 정량화

삽입된 가짜 구간의 부자연스러움을 4가지 고전 DSP 측정값으로 수치화했다(10ms 프레임).

측정값	정의	가짜 구간에서 기대
RMS 에너지	프레임 신호 세기(제공평균제공근)	삽입/볼륨정규화로 분포가 달라짐
ZCR	영교차율: 부호가 바뀌는 빈도	무성·고주파 성분 차이
Spectral centroid	스펙트럼 무게중심 주파수	보코더로 고주파 디테일 감소 → 낮아짐
Spectral flux	프레임간 스펙트럼 변화량	이음새/합성음에서 불연속 → 증가

4.1 부분스푸핑 300개 발화 평균 (프레임 단위)

측정값	bonafide	spoof	차이
RMS 에너지	0.024	0.015	-36.7%
ZCR	0.147	0.120	-18.1%
Spectral centroid (Hz)	1810	1708	-5.7%
Spectral flux	0.077	0.082	+6.9%

해석: 가짜 구간은 평균적으로 에너지·ZCR·중심주파수가 낮고 불연속(flux)이 높다 → 단서는 실재한다. 다만 차이가 5~37%로 크지 않고 발화마다 방향이 흔들린다(예: 그림 3의 한 발화는 가짜 RMS가 오히려 높음). 즉 '단일 임계값'으로는 부족하고, 스펙트럼 패턴 전체를 학습하는 분류기가 필요하다.

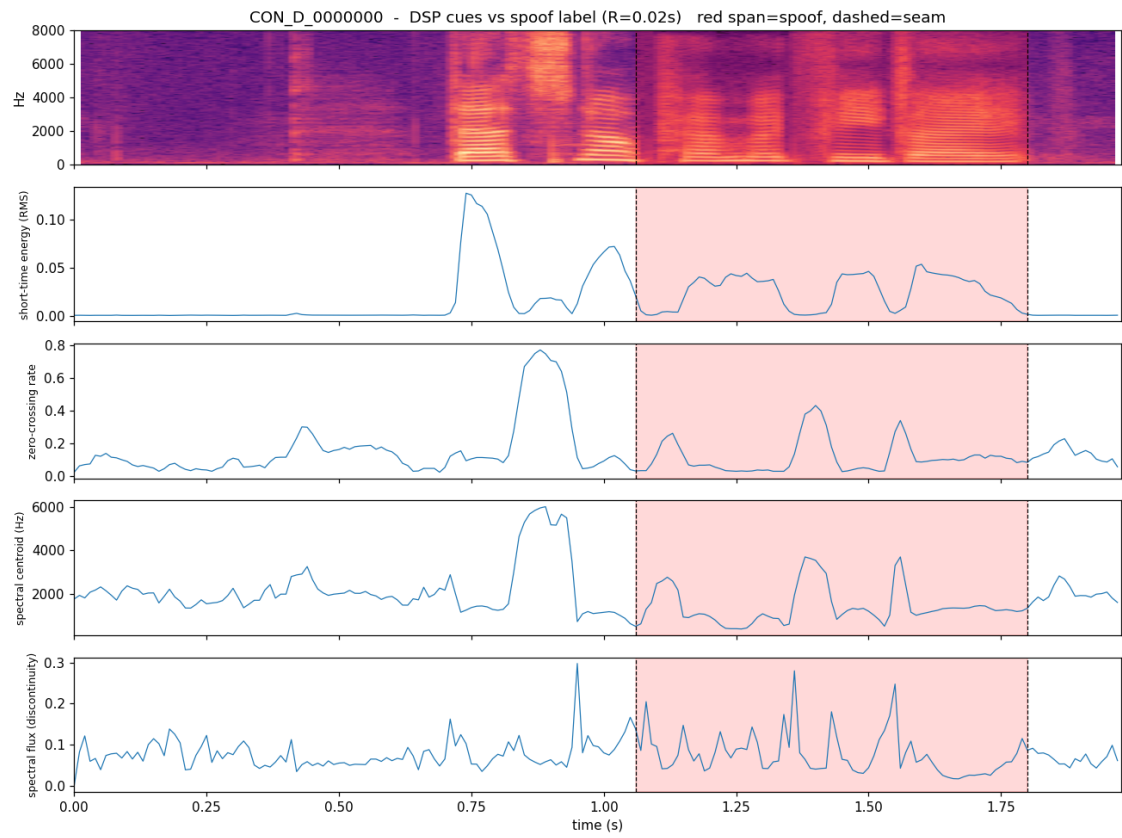


그림 3. DSP 측정값 vs 가짜 구간(빨강)·이음새(점선). flux가 이음새에서 튀.

5. 사용 피쳐 — LFCC / d·dd / STFT

피쳐	차원	무엇을 보는가	왜 PS에 유효한가
LFCC	40	선형 필터뱅크 캡스트럼	선형(비-멜) 스케일로 고주파·포먼트 디테일 보존
LFCC+d+dd	120	LFCC + 1·2차 시간미분	합성음의 부자연한 시간 동특성·이음새 포착
STFT-spec	514	로그-크기 STFT(257빈)	캡스트럼 압축 없이 전체 스펙트럼 질감

분석 프레임 32ms/홉 10ms로 추출 후, 세그먼트 라벨 해상도(0.16s) 윈도우로 mean+std 풀링하여 라벨과 1:1 정렬한다(각 차원은 평균+표준편차).

6. 방법 — 단순 로지스틱 회귀 베이스라인

- 국소화 = 윈도우 단위 이진분류(가짜 vs 진본). 피쳐별 LogisticRegression(class_weight=balanced) 이 윈도우마다 P(spoof)를 예측 -> 시간축 확률곡선.
- 탐지 = 발화 점수 = 윈도우 P(spoof)의 최댓값. 가짜 윈도우가 하나라도 있으면 spoof.
- 분할 = 발화 단위 70/30(GroupShuffleSplit) -> 같은 발화가 train/test에 섞이지 않음.

강조: 이는 의도적으로 단순한 베이스라인이다. 목적은 'DSP 특징으로 PS 단서가 분리되는가'의 확인이며, 전용 DSP 모델/딥러닝 CM은 다음 단계다.

7. 평가 지표 설명 — 기존 지표 vs 우리 지표

EER(Equal Error Rate, 동일오류율): 오수락률(FAR)과 오거부율(FRR)이 같아지는 지점의 오류율. 낮을수록 좋다(생체/스푸핑 탐지의 표준 지표).

지표	용도	정의/설명	출처
Utterance EER	탐지	발화 단위 진짜/가짜 EER (논문 권장)	metric/UtteranceEER.py
Segment EER	국소화	구간 단위(여러 해상도) EER	metric/SegmentEER.py
Range EER	국소화	구간 범위 기반 EER(pyannote), 권장 지표	metric/RangeEER.py
(우리) window EER	국소화	0.16s 윈도우 자체 EER (간이 지표)	experiment.py

차이/한계: 우리의 window-EER은 자체 정의 간이 지표라 논문/리더보드 수치와 '직접 비교 불가'다. 공식 비교를 위해서는 metric/cal_EER.sh 포맷으로 점수를 출력해 Utterance EER / Range EER로 재평가해야 한다(다음 단계).

8. 실험 결과

8.1 피처별 성능 (dev 1,500 발화, 32,800 윈도우)

피처	win-EER%	win-AUC	win-F1	utt-EER%	utt-AUC
LFCC	22.36	0.842	0.756	21.31	0.878
LFCC+d+dd	21.21	0.861	0.768	20.64	0.881
STFT-spec	16.94	0.905	0.814	9.40	0.971

win-* = 국소화(윈도우), utt-* = 탐지(발화). d 추가가 LFCC를 일관되게 개선하고, STFT 스펙트로그램이 최고. 즉 탐지 성능은 사실상 STFT가 견인한다.

8.2 탐지 난이도 분해 — 쉬운(완전가짜) vs 어려운(부분가짜)

피처	bona vs 전체	bona vs 완전가짜	bona vs 부분가짜
LFCC	21.3 / 0.878	13.5 / 0.939	24.0 / 0.855
LFCC+d+dd	20.6 / 0.881	12.2 / 0.940	24.2 / 0.859
STFT-spec	9.4 / 0.971	5.8 / 0.990	10.7 / 0.964

값 = EER% / AUC. 헤드라인 9.4%는 쉬운 완전가짜 덕을 보지만, 현실적 위협인 부분가짜도 STFT로 EER 10.7% / AUC 0.96 → 단순 회귀치고 유의미. (참고: 딥러닝 SOTA는 발화 EER 0.5~4%.)

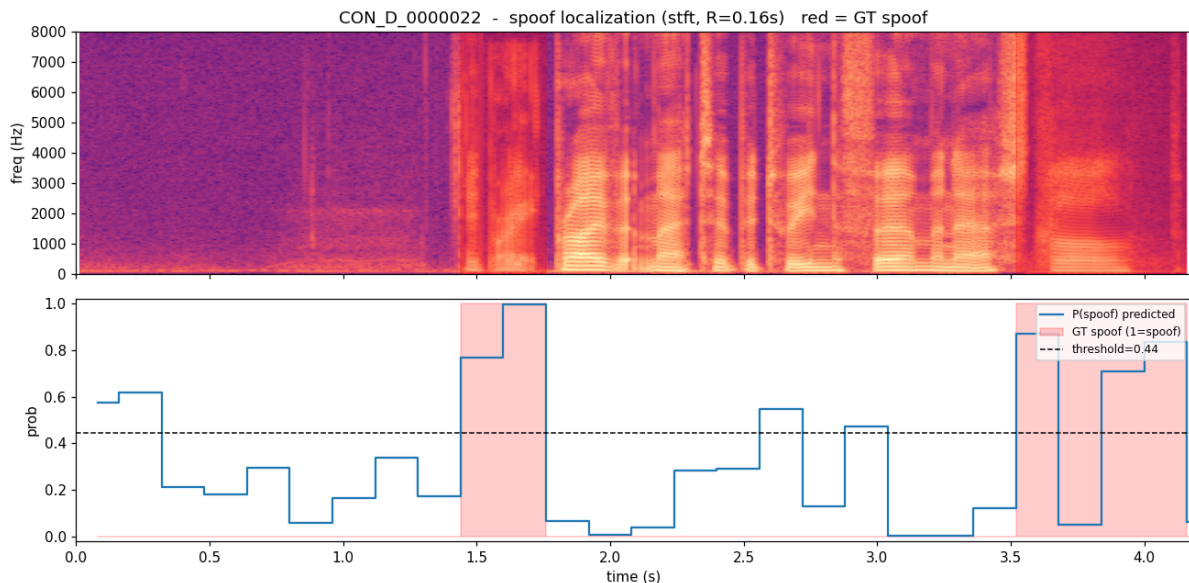


그림 4. 국소화 예시(STFT). 예측 P(spoof)가 정답 가짜 구간(빨강)에서 임계값 위로 상승.

9. 한계 및 다음 단계

- 지표: 공식 Range EER / Utterance EER(metric/ 코드) 연동해 논문과 같은 잣대로 재평가.
- 데이터: 현재 dev 내부 70/30 분할 → 공식 train으로 학습 후 dev/eval 평가(수치 낙관 편향 제거).
- 모델: 단순 로지스틱 회귀 → 전용 DSP 특징 + GMM/LightGBM/CNN, 점수 스무딩으로 국소화 개선.
- 현재 가장 약한 곳은 탐지가 아니라 국소화(window EER 16.9%).

요약: 데이터·라벨 검증 → 해상도/타입 분석 → DSP 단서 정량화 → 3개 피처 단순회귀 베이스라인까지 완료. 'DSP로 PS 단서가 분리된다'를 확인했고, 다음은 공식지표 연동과 모델 고도화.